

In [30]: *# Step 1: Imports and setting up the environment*

```
import os
import numpy as np
import pandas as pd
from PIL import Image

import torch
import torch.nn as nn
import torchvision.models as models
import torchvision.transforms as T

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, classification_report, roc_auc_score

# Detect CPU or GPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

Using device: cpu

In [32]: *import pandas as pd*

```
# 1) Read the NDJSON into a DataFrame
df = pd.read_json("yelp_photos/photos.json", lines=True)

# 2) Write out to CSV (no index column)
df.to_csv("yelp_photos/photos.csv", index=False)

print("CSV file created successfully!")
```

CSV file created successfully!

In [33]: *# 2.1) Load the full CSV*

```
csv_file = "yelp_photos/photos.csv"
df = pd.read_csv(csv_file)
print("Total rows in CSV:", len(df))

# Randomly sample 1000
df_sample = df.sample(n=30000, random_state=42).copy()
```

```
df_sample.reset_index(drop=True, inplace=True)
print("Sampled rows:", len(df_sample))
```

Total rows in CSV: 200100

Sampled rows: 30000

```
In [34]: # Create a column for image paths
def get_image_path(photo_id, folder="yelp_photos/photos"):
    # Adjust the folder or extension if your images are different
    return os.path.join(folder, f"{photo_id}.jpg")

df_sample["img_path"] = df_sample["photo_id"].apply(get_image_path)
df_sample.head()
```

```
Out[34]:
```

	photo_id	business_id	caption	label	img_path
0	k_PSngRS22mSA1MypwrjPg	DzzVSYXadZ1_XgfGz_Loyw	Chocolate Croissant	food	yelp_photos/photos\k_PSngRS22mSA1MypwrjPg.jpg
1	D_94KivwVgitkzFlgE_KcQ	Xdzir62WKISzeu4PMQtIBA	NaN	food	yelp_photos/photos\D_94KivwVgitkzFlgE_KcQ.jpg
2	Hf39P7_G_eRCqfVwvMDV6g	z0HzwNBmx_BgdiYI4hLk3g	Happy Anniversary	drink	yelp_photos/photos\Hf39P7_G_eRCqfVwvMDV6g.jpg
3	agxl4sABeRXwjLL506KMrQ	HzRSWmNxcEVQGrr1tun25w	Frozen Puccino	food	yelp_photos/photos\agxl4sABeRXwjLL506KMrQ.jpg
4	7cZ0MREN2TwAAX4nnirQIA	aj0urA2r2WlqZKufeB5dpw	Double Cheeseburger	food	yelp_photos/photos\7cZ0MREN2TwAAX4nnirQIA.jpg

```
In [38]: # Step 3: First split off 20% test, then split train_temp into train/val

train_temp, test_df = train_test_split(
    df_sample,
    test_size=0.2,           # 20% test
    stratify=df_sample["label"],
    random_state=42
)

train_df, val_df = train_test_split(
    train_temp,
    test_size=0.2,           # 20% of the 80% => 16% total
```

```

    stratify=train_temp["label"],
    random_state=42
)

print("Train size:", len(train_df))
print("Val size: ", len(val_df))
print("Test size: ", len(test_df))

```

Train size: 19200

Val size: 4800

Test size: 6000

In [40]: *# Step 4: Load ResNet18, remove final layer, define transforms*

```

resnet = models.resnet18(pretrained=True)
resnet.eval()
resnet = resnet.to(device)

# Remove the final FC layer to get 512-dim output
feature_extractor = nn.Sequential(*list(resnet.children())[:-1])
feature_extractor = feature_extractor.to(device)

# Image transformations (resize, tensor, normalize)
transform = T.Compose([
    T.Resize((224, 224)),
    T.ToTensor(),
    T.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

```

C:\Users\gaura\anaconda3\Lib\site-packages\torchvision\models_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, please use 'weights' instead.

warnings.warn(

C:\Users\gaura\anaconda3\Lib\site-packages\torchvision\models_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=ResNet18\_Weights.IMAGENET1K\_V1`. You can also use `weights=ResNet18\_Weights.DEFAULT` to get the most up-to-date weights.

warnings.warn(msg)

In [42]: *# Step 5: Functions to extract features from one image, and to build a feature dataset*

```
def extract_features(img_path, model, transform, device):  
    """  
    Opens and transforms an image, then extracts a 512-dim feature vector from ResNet.  
    """  
    image = Image.open(img_path).convert("RGB")  
    tensor = transform(image).unsqueeze(0).to(device)  
    with torch.no_grad():  
        feats = model(tensor) # shape [1, 512, 1, 1]  
    return feats.cpu().numpy().flatten()  
  
def build_feature_dataset(df, model, transform, device):  
    """  
    Loops over rows, extracts features for each image, returns (X, y).  
    Skips corrupted or missing files gracefully.  
    """  
    X, y = [], []  
    skipped = 0  
  
    for _, row in df.iterrows():  
        path = row["img_path"]  
        label = row["label"]  
  
        # Check if file is present and non-empty  
        if not os.path.exists(path) or os.path.getsize(path) == 0:  
            skipped += 1  
            continue  
  
        try:  
            feats = extract_features(path, model, transform, device)  
            X.append(feats)  
            y.append(label)  
        except Exception as e:  
            print(f"Skipping {path}, error: {e}")  
            skipped += 1  
  
    print(f"Skipped {skipped} corrupted or missing files.")  
    return np.array(X), np.array(y)
```

```
In [44]: # Step 6: Extract features from train, val, and test

X_train, y_train = build_feature_dataset(train_df, feature_extractor, transform, device)
X_val, y_val = build_feature_dataset(val_df, feature_extractor, transform, device)
X_test, y_test = build_feature_dataset(test_df, feature_extractor, transform, device)

print("X_train shape:", X_train.shape, "y_train shape:", y_train.shape)
print("X_val shape:", X_val.shape, "y_val shape:", y_val.shape)
print("X_test shape:", X_test.shape, "y_test shape:", y_test.shape)
```

```

Skipping yelp_photos/photos\E7Wpzn-1fCnVJ8_zKpecPQ.jpg, error: cannot identify image file 'yelp_photos/photos\E7Wpzn-1fCnVJ8_zKpecPQ.jpg'
Skipping yelp_photos/photos\RhC7TNmFvbR9GWr1r15dsA.jpg, error: cannot identify image file 'yelp_photos/photos\RhC7TNmFvbR9GWr1r15dsA.jpg'
Skipping yelp_photos/photos\NKEFWvRriK-LvagPz2QRxw.jpg, error: cannot identify image file 'yelp_photos/photos\NKEFWvRriK-LvagPz2QRxw.jpg'
Skipping yelp_photos/photos\W94rrCn005K1lkfD26m4tw.jpg, error: cannot identify image file 'yelp_photos/photos\W94rrCn005K1lkfD26m4tw.jpg'
Skipping yelp_photos/photos\1MOGQBWogR8oJr1WgERi9g.jpg, error: cannot identify image file 'yelp_photos/photos\1MOGQBWogR8oJr1WgERi9g.jpg'
Skipping yelp_photos/photos\CBxmBYD_5CXIL_F-2PDqmA.jpg, error: cannot identify image file 'yelp_photos/photos\CBxmBYD_5CXIL_F-2PDqmA.jpg'
Skipping yelp_photos/photos\yFjqHyOaNFwzIwTV8EE9hg.jpg, error: cannot identify image file 'yelp_photos/photos\yFjqHyOaNFwzIwTV8EE9hg.jpg'
Skipping yelp_photos/photos\pW1IPuTdLIUB61goirbXaA.jpg, error: cannot identify image file 'yelp_photos/photos\pW1IPuTdLIUB61goirbXaA.jpg'
Skipping yelp_photos/photos\JZZ716oX6_MqH6L_MkWK-A.jpg, error: cannot identify image file 'yelp_photos/photos\JZZ716oX6_MqH6L_MkWK-A.jpg'
Skipped 9 corrupted or missing files.
Skipping yelp_photos/photos\rIhUkEmP-j4NcQVW3YuPYQ.jpg, error: cannot identify image file 'yelp_photos/photos\rIhUkEmP-j4NcQVW3YuPYQ.jpg'
Skipping yelp_photos/photos\feUGw0P5by0q4U40C77tyQ.jpg, error: cannot identify image file 'yelp_photos/photos\feUGw0P5by0q4U40C77tyQ.jpg'
Skipping yelp_photos/photos\7xcWPjcE4mxoQ1AjvvKJZg.jpg, error: cannot identify image file 'yelp_photos/photos\7xcWPjcE4mxoQ1AjvvKJZg.jpg'
Skipped 3 corrupted or missing files.
Skipping yelp_photos/photos\qMlGILrsrzhMDxajNYiyIA.jpg, error: cannot identify image file 'yelp_photos/photos\qMlGILrsrzhMDxajNYiyIA.jpg'
Skipped 1 corrupted or missing files.
X_train shape: (19191, 512) y_train shape: (19191,)
X_val shape: (4797, 512) y_val shape: (4797,)
X_test shape: (5999, 512) y_test shape: (5999,)

```

```

In [45]: # Step 7: k-NN training (simple) and validation evaluation

knn = KNeighborsClassifier(n_neighbors=5, metric='euclidean')
knn.fit(X_train, y_train)

val_accuracy = knn.score(X_val, y_val)
print("Validation Accuracy (k=5, Euclidean):", val_accuracy)

```

Validation Accuracy (k=5, Euclidean): 0.9407963310402335

```
In [46]: # Validate with confusion matrix & classification report
y_val_pred = knn.predict(X_val)
cm_val = confusion_matrix(y_val, y_val_pred)
print("\nConfusion Matrix (Val):\n", cm_val)

print("\nClassification Report (Val):")
print(classification_report(y_val, y_val_pred))

# AUC for multi-class (one-vs-rest, macro average)
y_val_proba = knn.predict_proba(X_val)
val_auc = roc_auc_score(y_val, y_val_proba, multi_class='ovr', average='macro')
print("Validation AUC:", val_auc)
```

Confusion Matrix (Val):

```
[[ 303   40   26    0    2]
 [  34 2535   33    3    1]
 [   10   32 1274    0   25]
 [    1    1    7   28    1]
 [    1    3   64    0  373]]
```

Classification Report (Val):

	precision	recall	f1-score	support
drink	0.87	0.82	0.84	371
food	0.97	0.97	0.97	2606
inside	0.91	0.95	0.93	1341
menu	0.90	0.74	0.81	38
outside	0.93	0.85	0.88	441
accuracy			0.94	4797
macro avg	0.92	0.86	0.89	4797
weighted avg	0.94	0.94	0.94	4797

Validation AUC: 0.9647957704617578

```
In [47]: # Step 8: Validation set evaluation

val_acc = knn.score(X_val, y_val)
print("Validation Accuracy:", val_acc)

y_val_pred = knn.predict(X_val)
cm_val = confusion_matrix(y_val, y_val_pred)
```

```

print("\nConfusion Matrix (Val):\n", cm_val)

print("\nClassification Report (Val):")
print(classification_report(y_val, y_val_pred))

# AUC (multiclass, one-vs-rest)
y_val_proba = knn.predict_proba(X_val)
val_auc = roc_auc_score(y_val, y_val_proba, multi_class='ovr', average='macro')
print("Validation AUC (macro, OVR):", val_auc)

```

Validation Accuracy: 0.9407963310402335

Confusion Matrix (Val):

```

[[ 303   40   26    0    2]
 [  34 2535   33    3    1]
 [  10   32 1274    0   25]
 [    1    1    7   28    1]
 [    1    3   64    0  373]]

```

Classification Report (Val):

	precision	recall	f1-score	support
drink	0.87	0.82	0.84	371
food	0.97	0.97	0.97	2606
inside	0.91	0.95	0.93	1341
menu	0.90	0.74	0.81	38
outside	0.93	0.85	0.88	441
accuracy			0.94	4797
macro avg	0.92	0.86	0.89	4797
weighted avg	0.94	0.94	0.94	4797

Validation AUC (macro, OVR): 0.9647957704617578

In [48]: *# Step 8: (Optional) Hyperparameter tuning with GridSearchCV*

```

param_grid = {
    'n_neighbors': [3, 5],
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan']
}

```



```
knn_model = KNeighborsClassifier()
grid_search = GridSearchCV(
    knn_model,
    param_grid,
    cv=3,
    scoring='accuracy',
    verbose=1
)
```

```
In [49]: grid_search.fit(X_train, y_train)

print("Best Params:", grid_search.best_params_)
print("Best CV accuracy:", grid_search.best_score_)

best_knn = grid_search.best_estimator_

# Evaluate on validation
val_accuracy_best = best_knn.score(X_val, y_val)
print("Validation Accuracy (best params):", val_accuracy_best)
```

Fitting 3 folds for each of 8 candidates, totalling 24 fits


```

^^^^^^^^^^^^^^^^^^^^
File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\metrics\_scorer.py", line 87, in _cached_call
    result, _ = _get_response_values(
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\utils\_response.py", line 210, in _get_response_values
    y_pred = prediction_method(X)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\neighbors\_classification.py", line 259, in predict
    probabilities = self.predict_proba(X)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\neighbors\_classification.py", line 343, in predict_proba
    probabilities = ArgKminClassMode.compute(
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\metrics\_pairwise_distances_reduction\_dispatcher.py", line 604, in compute
    unique_Y_labels=np.array(unique_Y_labels, dtype=np.intp),
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'drink'

warnings.warn(
C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\model_selection\_validation.py:993: UserWarning: Scoring failed. The score on this train-test partition for these parameters will be set to nan. Details:
Traceback (most recent call last):
  File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\model_selection\_validation.py", line 982, in _score
    scores = scorer(estimator, X_test, y_test, **score_params)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\metrics\_scorer.py", line 253, in __call__
    return self._score(partial(_cached_call, None), estimator, X, y_true, **kwargs)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\metrics\_scorer.py", line 345, in _score
    y_pred = method_caller(
    ^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\metrics\_scorer.py", line 87, in _cached_call
    result, _ = _get_response_values(
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\utils\_response.py", line 210, in _get_response_values
    y_pred = prediction_method(X)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\neighbors\_classification.py", line 259, in predict
    probabilities = self.predict_proba(X)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\neighbors\_classification.py", line 343, in predict_proba

```

```

probabilities = ArgKminClassMode.compute(
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\metrics\_pairwise_distances_reduction\_dispatcher.py", line 604, in compute
    unique_Y_labels=np.array(unique_Y_labels, dtype=np.intp),
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'drink'

warnings.warn(
C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\model_selection\_validation.py:993: UserWarning: Scoring failed. The score on this train-test partition for these parameters will be set to nan. Details:
Traceback (most recent call last):
  File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\model_selection\_validation.py", line 982, in _score
    scores = scorer(estimator, X_test, y_test, **score_params)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\metrics\_scorer.py", line 253, in __call__
    return self._score(partial(_cached_call, None), estimator, X, y_true, **kwargs)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\metrics\_scorer.py", line 345, in _score
    y_pred = method_caller(
    ^^^^^^^^^^^^^^^^^
  File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\metrics\_scorer.py", line 87, in _cached_call
    result, _ = _get_response_values(
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\utils\_response.py", line 210, in _get_response_values
    y_pred = prediction_method(X)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\neighbors\_classification.py", line 259, in predict
    probabilities = self.predict_proba(X)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\neighbors\_classification.py", line 343, in predict_proba
    probabilities = ArgKminClassMode.compute(
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\metrics\_pairwise_distances_reduction\_dispatcher.py", line 604, in compute
    unique_Y_labels=np.array(unique_Y_labels, dtype=np.intp),
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'drink'

warnings.warn(
C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\model_selection\_validation.py:993: UserWarning: Scoring failed. The score on this train-test partition for these parameters will be set to nan. Details:

```

```

Traceback (most recent call last):
  File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\model_selection\_validation.py", line 982, in _score
    scores = scorer(estimator, X_test, y_test, **score_params)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\metrics\_scorer.py", line 253, in __call__
    return self._score(partial(_cached_call, None), estimator, X, y_true, **kwargs)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\metrics\_scorer.py", line 345, in _score
    y_pred = method_caller(
    ^^^^^^^^^^^^^^^^^
  File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\metrics\_scorer.py", line 87, in _cached_call
    result, _ = _get_response_values(
    ^^^^^^^^^^^^^^^^^
  File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\utils\_response.py", line 210, in _get_response_values
    y_pred = prediction_method(X)
    ^^^^^^^^^^^^^^^^^
  File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\neighbors\_classification.py", line 259, in predict
    probabilities = self.predict_proba(X)
    ^^^^^^^^^^^^^
  File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\neighbors\_classification.py", line 343, in predict_proba
    probabilities = ArgKminClassMode.compute(
    ^^^^^^^^^^^^^
  File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\metrics\_pairwise_distances_reduction\_dispatcher.py", line 604, in compute
    unique_Y_labels=np.array(unique_Y_labels, dtype=np.intp),
    ^^^^^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'drink'

warnings.warn(
C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\model_selection\_validation.py:993: UserWarning: Scoring failed. The score on this train-test partition for these parameters will be set to nan. Details:
Traceback (most recent call last):
  File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\model_selection\_validation.py", line 982, in _score
    scores = scorer(estimator, X_test, y_test, **score_params)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\metrics\_scorer.py", line 253, in __call__
    return self._score(partial(_cached_call, None), estimator, X, y_true, **kwargs)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\metrics\_scorer.py", line 345, in _score
    y_pred = method_caller(
    ^^^^^^^^^^^^^^^^^
  File "C:\Users\gaura\anaconda3\Lib\site-packages\sklearn\metrics\_scorer.py", line 87, in _cached_call

```



```
test_auc = roc_auc_score(y_test, y_test_proba, multi_class='ovr', average='macro')
print("Test AUC:", test_auc)
```

Test Accuracy: 0.941656942823804

Confusion Matrix (Test):

```
[[ 374   43   44    1    2]
 [  35 3176   47    3    0]
 [   8   30 1605    4   29]
 [   0    2    7   37    1]
 [   0    3   90    1  457]]
```

Classification Report (Test):

	precision	recall	f1-score	support
drink	0.90	0.81	0.85	464
food	0.98	0.97	0.97	3261
inside	0.90	0.96	0.93	1676
menu	0.80	0.79	0.80	47
outside	0.93	0.83	0.88	551
accuracy			0.94	5999
macro avg	0.90	0.87	0.88	5999
weighted avg	0.94	0.94	0.94	5999

Test AUC: 0.9775893816817867

```
In [62]: # Save the best model
from joblib import dump
model_filename = "best_knn.joblib"
dump(best_knn, model_filename)
print(f"Best model saved to {model_filename}")
```

Best model saved to best_knn.joblib

In []: